

TOPIC NAME: Chapter 1

DAY: _____ Azmari Sultana

TIME: _____ DATE: / /

Introduction to Computer Interfacing

Computer Interfacing: An interface is a shared boundary where two or more separate components (I/O devices, Memory, etc.) of a computer system exchange information.

Types of Computer Interfaces:

Software Interface: Interaction between programs or modules.

(e.g., API)

Hardware Interface: Communication between hardware components

(e.g., CPU, I/O devices and memory).

User Interface (UI): Where humans interact with computers

(e.g., GUI or command line)

Network Interface: Where systems communicate over networks

(e.g., network cards)

Examples:

Human-Machine Interface (HMI): Like touch screens or keyboards.

Peripheral Interfaces: Communication with devices like printers, mice, or storage.

Software Interfaces: APIs allowing programs to interact (e.g., REST, SOAP).

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /

Peripheral Communication Links:

- Peripherals connected to a computer require special communication links to interface with the CPU.
- Input-output interfaces are hardware components that manage data transfers between the CPU and peripherals.

CPU vs Peripheral Devices: key Differences

Feature	CPU (Internal)	Peripheral devices (External)	Requirement for Interface
Nature of operation	Electronic: Operates purely on digital signals at extremely high speeds.	Electromagnetic/Electromechanical: Often involves physical movement (e.g., motors, sensors).	Signal conversion: Translates physical/mechanical actions into digital data.
Data Transfer Rate	Very fast: High-speed internal bus communication.	Slow to moderate: Limited by physical components or external connection speeds.	Synchronization: Uses buffers and handshaking signals to match speeds.
Data Format	Word-oriented: Standardized internal bit-lengths (e.g., 32 bit or 64 bit).	Varies: Can be serial, parallel, or use different character codes (e.g., ASCII).	Format conversion: Restructures data so the CPU can process it.
Control & Mode	Centralized: Controlled by the system clock and instruction set.	Autonomous: Each has its own operating mode and timing.	Device management: Ensures peripherals don't conflict or interrupt the CPU unnecessarily.

TOPIC NAME : _____

DAY : _____

TIME : _____ DATE : / /

Peripheral Devices

A peripheral is a device connected to a host computer that expands its capabilities but is not part of the core architecture. It can be partially or completely dependent on the host for operation.

Types of Peripheral Devices:

Input devices: Devices that send data to the computer
(e.g., keyboard, mouse).

Output devices: Devices that receive data from the computer
(e.g., monitor, printer).

Input/Output devices: Devices that both send and receive data
(e.g., external hard drives).

How does a computer work?

Input:

- i) Data entered through devices like keyboard, Mouse, Scanner
- ii) Can be analog (e.g., microphone) or digital (e.g., keyboard).

Processing:

- i) Handled by the CPU, the computer's brain
- ii) Performs calculations and controls tasks
- iii) RAM (memory) provides temporary storage for fast access.

GOOD LUCK™

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /

Output:

- i) Results displayed on output devices such as Monitor, Printer, Speakers.
- ii) Forms of output: visual, printed, on audio.

Storage:

- i) Processed information saved in HDD/SSD, cloud storage.
- ii) Stored as files, images, on databases.

Software hierarchy of a computer system

User: The person interacting with the computer, using various applications for tasks such as word processing, spreadsheets or graphics.

Application software:

- Programs used by the user to perform specific tasks, including word processing, Database management, Graphics.

Software system software:

- Manages the overall functioning of the computer:
 - i) Operating system (OS): Controls hardware and provides services to application software.

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /

ii) Device Drivers : Facilitate communication between the OS and hardware devices.

iii) Utility programs : perform maintenance tasks like virus scanning and disk cleanup.

Hardware : The physical components of the computer (e.g., monitor, keyboard, CPU, speakers), along with peripheral devices like printers.

Bus Architecture : A bus architecture is a system of pathways used for communication between components inside a computer. It transfers data, addresses and control signals.

key components:

i) CPU : The central processing unit that sends and receives data.

ii) Memory : Where data is stored for quick access.

iii) I/O devices : Devices like keyboards, printers etc., that

interact with the CPU.

Unique addresses

Each I/O device (e.g., monitor, keyboard) has its own

unique address.

TOPIC NAME: _____

DAY: _____

TIME: _____

DATE: / /

System Bus: A set of wires that interconnects all the components (subsystems) of a computer system.

Data flow:

i) Source component: Outputs data onto the bus.

ii) Destination component: Inputs data from the bus.

Bus Hierarchy:

i) Address, Data, and Control Buses: Used to access memory and I/O controllers.

ii) I/O controller: Second set of buses connects the I/O controller to attached devices (e.g., peripherals)

Example: PCI Bus: A common local bus example used in many computer systems.

Address Bus: The CPU reads/writes data by addressing a unique location. The location is output onto the address bus, and the memory uses it to access the proper data.

Unique addresses:

- Each I/O device (e.g., monitor, keyboard) has its own unique address.

TOPIC NAME: _____

DAY: _____

TIME: _____

DATE: / /

- CPU places the address on the address bus when accessing an I/O device, and the connected device acts accordingly.

Uni-directional:

- Data flows only from the CPU to the address bus.

- The CPU never reads data from the address bus.

Data Bus: Transfers actual data between the CPU, memory, and I/O devices.

Data Transfer Process:

- CPU Fetches:

i) CPU outputs address on the address bus.

ii) Memory outputs data onto the data bus.

iii) CPU reads the data from the data bus.

- CPU Writes:

i) CPU outputs address on the address bus.

ii) CPU outputs data onto the data bus.

iii) Memory reads the data and stores it.

Bidirectional: Data can flow to and from the CPU.

TOPIC NAME: _____

DAY: _____

TIME: _____

DATE: / /

Control Bus: The control bus is a collection of individual control signals, unlike address and data buses that transmit n-bit values.

Signals:

i) **Read/Write Signals:** Whether data is being read into or written out of the CPU.

ii) **Memory/I/O signals:** Whether the CPU is accessing memory or an I/O device.

iii) **Ready Signal:** Whether the I/O device or memory is ready for data transfer.

Unidirectional: Control signals can flow to and from the CPU for synchronization between the CPU and other components like memory and I/O devices.

It can be bidirectional, but mostly unidirectional.

TOPIC NAME : _____

DAY : _____

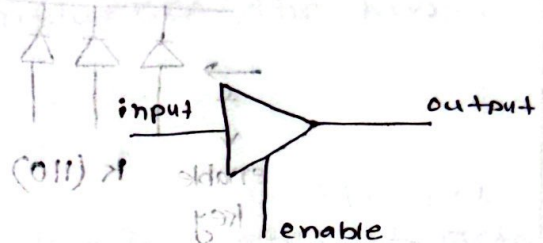
TIME : _____

DATE : / /

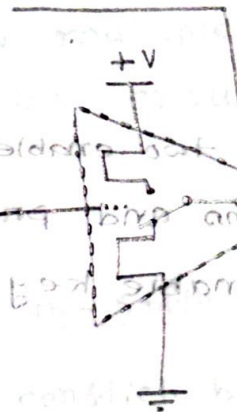
Tri-state Logic : This state logic allows a bus to be shared by multiple devices, but ensures that only one device can access the bus at any given time.

That means Multiple device can receive at a time,
Multiple device can't send at a time.

enable	input	output
0	X	Z
1	0	0
1	1	1



Enable



* Suppose, the data bus is 3 bit. There are Display (D), keyboard (K), Mouse (M), pendrive (P).

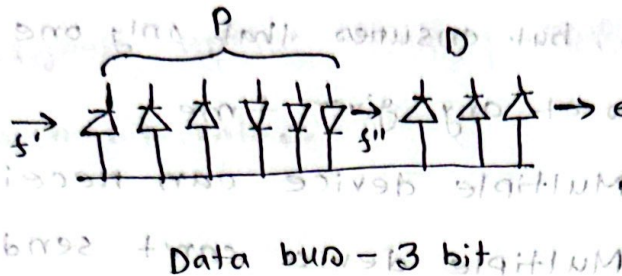
keyboard (K) } Bidirectional
 Mouse (M) } Unidirectional
 pendrive }

TOPIC NAME: _____

DAY: _____

TIME: _____

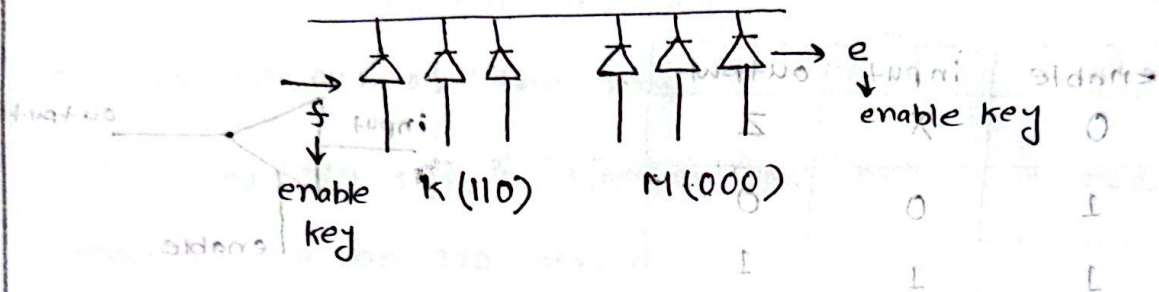
DATE: / /



Unidirectional $\rightarrow$ 2

4 bit, 128
keyboard $\rightarrow$ 3 bit

Bidirectional $\rightarrow$
existing bits multi-
plied by 2. (2x2)
pendrive $\rightarrow$ 3x2



Unidirectional buffers require only one enable pin, because data flows in a single direction. Example: keyboard has 1 enable key (f).

Bidirectional buffers require two enable controls to control data flow in both directions and prevent bus contention. Example: Pendrive has 2 enable key (f', f'').

When data is transferred from the keyboard to the display,

$f = 1, e = 0, f'' = 0, f' = 0, e' = 1$

When data is transferred from the keyboard to the display and pendrive,

$f = 1, e = 0, f'' = 0, f' = 1, e' = 1$

TOPIC NAME : _____

DAY : _____

TIME : _____ DATE : / /

Three states :

- i) Logic 0 : Represents a low voltage signal (e.g., 0V).
- ii) Logic 1 : Represents a high voltage signal (e.g., 5V).
- iii) High Impedance (Hi-Z) : This is the third state where the device is effectively disconnected from the circuit and does not influence the bus.

How Tri-stating works :

- When a device is in the Hi-Z state, it acts as though it is disconnected, preventing interference with other devices on the bus.
- This enables multiple devices to be connected to the same bus but ensures that only one device is active at a time.

Advantage of Tri-State Bus

- i) To resolve conflict between multiple sources driving the shared bus line.
- ii) To use the same port as both for input and output operation.

Note : Not all devices need to drive all bus lines.

- iii) It higher the bus fan out.

TOPIC NAME: _____

DAY: _____

TIME: _____

DATE: / /

Bus Interfacing

*** Bus Master:** The device that controls communication on the bus, determining who will speak and who will listen.

Bus master $\rightarrow$ CPU

Advanced buses can allow other devices to become bus masters when necessary.

Input/Output Operations:

- Read: Data moves into the bus master (input)
- Write: Data moves out from the bus master (output)

Bus lines:

Each bus has several lines dedicated to three main functions:

i) Addressing

ii) Data Transfer

iii) Control

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /

Bus Errors and Conflicts:

* Bus conflict: Occurs when multiple I/O devices try to use the bus simultaneously as output.

No conflict: If one device is used as output and others as input, no bus errors occur.

Causes of Bus conflicts/Errors:

- i) Multiple devices using the bus as output at the same time.
- ii) Wait states not being properly maintained (mismatch between speeds). Data can be lost or corrupted if we don't maintain wait states when required.
- iii) An I/O device causing a bus pin to get stuck at 1 or 0.
- iv) Glitches: Short-lived faults that may cause transient errors.

TOPIC NAME: _____

DAY: _____

TIME: _____

DATE: / /

Wait State: A time-out period when the CPU on bus lines is idle to account for components working at different speeds.

- Necessary when the CPU is faster than the memory or other components.
- Without proper wait states, synchronization issues can lead to bus errors.

* Glitches: A temporary fault that is self-correcting but can cause errors like signal transitions at the wrong time.

Example: In I2C bus systems, glitches can cause incorrect start commands to be detected, disrupting communication.

TOPIC NAME : _____

DAY : _____

TIME : _____ DATE : / /

Computer Ports

Computer ports are connection points (interfaces) on a computer that allow peripheral devices to connect and communicate with the computer hardware (especially the CPU and motherboard).

Based on how data is transferred, computer ports are mainly of two types:

i) Serial port

ii) Parallel port

Other types are: PS2, SCSI etc.

Parallel port

A parallel port transfers multiple bits of data simultaneously (typically 8 bits per clock cycle) using multiple communication lines. It is used for communication between a computer and peripheral devices such as printers. Parallel ports commonly use 25-pin or 36-pin connectors. The remaining lines are used for control signals, status signals and grounding, which help in data synchronization and communication.

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /

Bidirectional data transfer: A parallel port supports

bidirectional communication, meaning data can be transmitted between two devices in both directions.

For example: Device A can send data to Device B, and Device B can also send data to Device A.

However, this communication does not always occur simultaneously. Depending on the port mode, it can

operate in:

i) Half-duplex mode: Data can flow in both directions, but only one direction at a time.

ii) Full-duplex mode: Data can flow in both directions simultaneously.

* How to make a parallel port bidirectional?

- To make a parallel port bidirectional, the port must follow the IEEE 1284 standard, which allows data transmission in both directions between devices.

TOPIC NAME : _____

DAY : _____

TIME : _____ DATE : / /

* How to make a parallel port bidirectional and full duplex?

- To make a parallel port bidirectional and full duplex, the 8 data lines are divided into two equal groups instead of using all 8 lines for communication in one direction.

The first half of the lines (4 lines) are dedicated to sending data from device A to device B, and the second half (4 lines) are used to send data from device B to device A.

This allows both devices to transmit data simultaneously, meaning communication occurs at the same clock cycle (full duplex). However, since only 4 lines are used in each direction instead of 8, transmitting an 8-bit data value requires two clock cycles instead of one.

Enhancement modes:

The complete enhancement modes of parallel port are EPP and ECP, defined by the IEEE 1284 standard to enable high-speed, bidirectional and more efficient data communication.

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /

Serial Port

A serial port transfers data one bit at a time per clock cycle over a communication line. Standard serial ports often use a 9-pin connector.

Since it usually uses a single data line, communication is half-duplex. That means —

- Device A sends data to Device B, then stops.
- Device B sends data to Device A.

If the serial port uses two separate lines (one for sending, one for receiving), it can support full-duplex communication, allowing both devices to transmit data simultaneously.

In addition to the data line, the remaining pins are used for synchronization, control signals and grounding.

Nowadays, it has been replaced by ports like USB, VGA, and Ethernet for more efficient communication.

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /

Serial and Parallel Ports

All interface elements, from keyboard to mouse, eventually transport digital information from the external world to the CPU. There are two ways -

- i) Serial communication
- ii) Parallel communication

Serial ports are used for serial communication.
Parallel ports are used for parallel communication.

Serial Transmission:

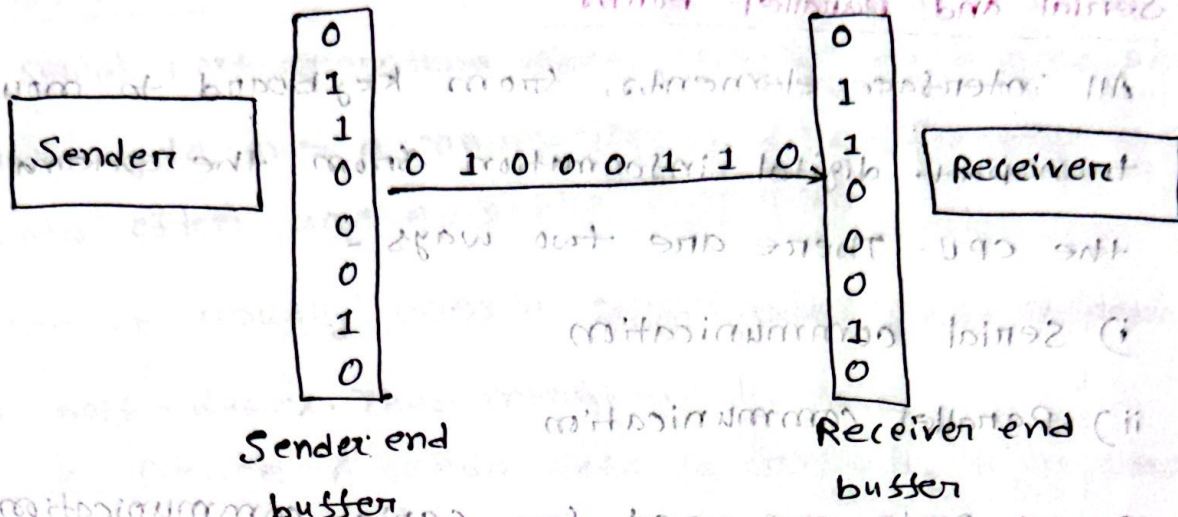
When data is sent or received using serial data transmission, the data bits are organized in a specific order, since they can only be sent one after another. The order of the data bits is important as it dictates how the transmission is organized when it is received. It is viewed as a reliable data transmission method, because a data bit is only sent if the previous data bit has already been received.

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /



Serial Data Transmission with Buffers

In serial transmission, both the sender and receiver devices have buffers to temporarily store data. These buffers help maintain synchronization, preserve the order of data and make communication more reliable.

Data Storage in Buffers:

- i) Data is first placed in the sender's buffer before transmission.
- ii) Bits are typically stored from MSB to LSB.
- iii) For example, if we want to send 8 bits, all 8 bits are loaded into the sender's buffer.

TOPIC NAME : _____

DAY : _____

TIME : _____ DATE : / /

Transmission process

- i) Data is sent one bit per clock cycle to the receiver device.
- ii) The receiver's buffer collects the incoming bits. Once all bits arrive, the complete data is transferred from the receiver buffer to the device for processing.

Advantages of using buffers

- i) Easier communication: Buffers smooth out timing differences between sender and receiver.
- ii) Synchronization: They help maintain the correct sequence of bits.
- iii) Data integrity: Using buffers allows verification of transmitted data.

Error Detection and Reliability

- i) If a bit is lost during transmission, the sender buffer still holds the original data.
- ii) The receiver can detect missing or corrupted bits by comparing the received data with the expected data.

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /

iii) If the received data does not match, an acknowledgement (Ack) error is triggered, and the sender retransmits the entire data packet.

When is serial transmission used to send data?

- serial transmission is normally used for long-distance data transfer. It is also used in cases where the amount of data being sent is relatively small. It ensures that data integrity is maintained as it transmits the data bits in a specific order, one after another. In this way, data bits are received in sync with one another.

Error Detection and Reliability

- i) If a bit is lost during transmission, the packet still holds the original data.
- ii) The receiver can detect missing or corrupted bits by comparing the received data with the expected data.

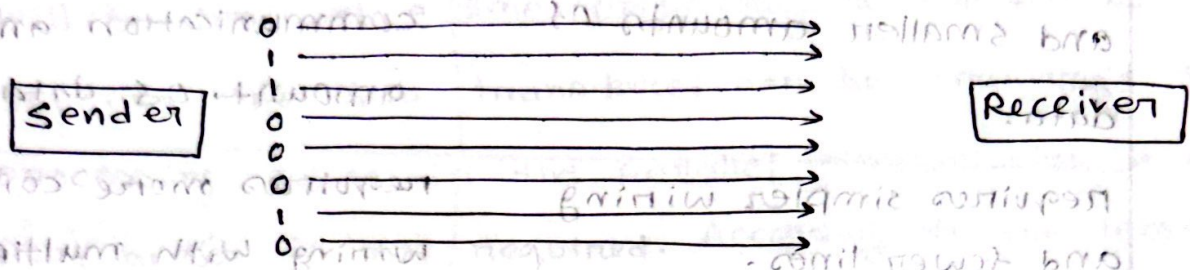
TOPIC NAME : _____

DAY : _____

TIME : _____ DATE : / /

Parallel Transmission

When data is sent using parallel data transmission, multiple data bits are transmitted over multiple channels at the same time. This means that data can be sent much faster than using serial transmission methods.



Simultaneous Transmission

of 8 bit data

This communication comes for rescue when speed rather than space is the main objective. The transfer of data is at high speed owing to the fact that no bus buffer is present.

Differences between Serial and Parallel Transmission

Serial Transmission	Parallel Transmission
Data is sent 1 bit per clock cycle.	Data is sent multiple bits per clock cycle.
It uses buffers at the sender and receiver end for synchronization and error checking.	It does not usually use buffers and data is sent directly.

TOPIC NAME: _____

DAY: _____

TIME: _____ DATE: / /

It is more secure and has a lower chance of crosstalk between lines.

It is less secure and has a higher chance of crosstalk between lines.

Serial transmission is suitable for long distance communication and smaller amounts of data.

Parallel transmission is suitable for short distance communication and large amount of data.

Requires simpler wiring and fewer lines.

Requires more complex wiring with multiple lines.

Lower in cost due to fewer wires.

Higher in cost due to multiple wires.

Parallel communication transfers 8, 16 or 32 bits one each clock pulse. Clearly this is much faster, but it requires many simultaneous data lines.

This is really the only choice for very fast processes, like sending data from the CPU to external Random Access Memory (RAM).

The number of data lines could quickly become ridiculous if you have many separate devices that communicate with the CPU.

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /

Why serial communication is slower than parallel?

For a 8bit data transfer in Serial communication one bit will be sent at a time. The entire data is first fed into the serial port buffer. From this buffer one bit will be sent at a time. Only after the last bit is received the data transferred can be forwarded for processing. While in the parallel communication a serial port buffer is not required. According to the length of the data, the number of bus lines are available plus a synchronization line for synchronized transmission of data. Thus we state that for the same frequency of data transmission, Serial communication is slower than parallel communication.

When parallel communication is used?

Parallel communication is used when:

- i) a large amount of data is being sent
- ii) the data being sent is time-sensitive
- iii) and the data needs to be sent quickly.

A scenario where parallel transmission is used to send data

TOPIC NAME: _____

DAY: _____

TIME: _____

DATE: / /

is **video streaming**. When a video is streamed to a viewer, bits need to be received quickly to prevent a video pausing or buffering. Video streaming also requires the transmission of large volumes of data. The data being sent is also time-sensitive as slow data streams result in poor viewer experience.

Why serial communication is preferred over parallel?

Serial communication is preferred over parallel communication especially for long-distance transmission. In serial communication, data is sent one bit at a time through a single line. This makes the system simpler, cheaper and less affected by noise and interference.

In parallel communication, multiple bits are sent at the same time using multiple wires. Although parallel communication can be faster over short distances, it becomes problematic over long distances. This is because different wires may have slightly different delays, causing synchronization errors and incorrect data reception.

TOPIC NAME: _____

DAY: _____

TIME: _____

DATE: / /

Another major problem in parallel communication is crosstalk. Crosstalk happens when signals from one wire interfere with signals in another wire, which can cause errors. Since parallel communication uses many wires, the chance of crosstalk and noise is higher.

Serial communication avoids these problems, because it uses only one data line. Devices like USART/UART help send and receive data correctly and maintain synchronization. As a result, serial communication has fewer errors, less noise and works reliably over long distances.

Examples

Parallel Transmission: An example of parallel transmission is the connection between a computer and a printer using a parallel port and cable. Most printers are placed close to the computer, usually within 6 meters (20 feet). Parallel transmission uses multiple wires to send data at the same time, which makes it faster. Since the distance is short, the extra cost of using more wires is acceptable, and the higher speed is beneficial.

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /

21 Serial transmission: An example of serial transmission is the connection between a computer and a modem using the RS-232 protocol. Although an RS-232 cable may contain up to 25 wires, only two wires are used to transmit actual data. The remaining wires are used for control signals. Serial transmission sends data one bit at a time in each direction. This method is more economical, especially when the computer and modem are far apart, because it uses fewer wires. In this case, lower cost and reliability are more important than speed.

Examples

and the highest speed is practical.

Microprocessor	Microcontroller
Used where intensive processing is required	Used where task is fixed and predefined
Only the CPU is in the chip. Memory, I/O port are connected externally	CPU, Memory, I/O port – all are connected on the same single chip
Higher Clock speed and external RAM used is also higher	Lower Clock speed and RAM used is also lower
The program for the microprocessor can be changed for different applications.	The program for the microcontroller is fixed once it is designed
Cost is comparatively higher	Cost is comparatively lower
Power consumption is higher	Power consumption is lower
Overall size of the system is large	Overall size of the system is smaller
Applications include personal computers	Applications include washing machines, camera etc.